Table of Contents

Student Lab Manual: Automating VLAN Configuration Using Python and NETCONF 2

Objectives 2

Background 2

Environment 2

Topology Description 3

Required Resources 4

Optional Resources 5

Lab Requirements 5

Instructions 6

Step 1: Verify NETCONF is Enabled 6

Step 2: Create Python Script for VLAN Configuration 6

Step 3: Execute the Configuration Script 8

Step 4: Create Verification Script 9

Step 5: Execute Verification Script 13

Step 6: Verify Using CLI 13

Expected Results 14

Troubleshooting Tips 14

Reflection Questions 15

Reflection Summary 15

Student Lab Manual: Automating VLAN Configuration Using Python and NETCONF

Objectives

In this lab, you will leverage Python code using NETCONF protocol to push/configure VLANs to a Cisco CSR1000v router. You will automate pushing the configuration to the device using Python scripts. You will also automate retrieval and verification of the device configuration. Upon completion of this lab, you will be able to:

1. Understand the purpose of network automation and its advantages over using a CLI to configure network devices
2. Connect to a Cisco CSR1000v router using Python and NETCONF
3. Create VLAN subinterfaces and configure them with dot1Q encapsulation using automation (XML/YANG)
4. Run Python code instead of repeatedly typing commands
5. Validate that your configuration was applied by retrieving it with automation (NETCONF)
6. Validate that your configuration is correct using CLI commands (optional)

Background

Manual network configuration involves typing commands into a device via its CLI. This works, however; it can become time consuming when your network starts expanding.

Cisco router platforms like the CSR1000v use subinterfaces with 802.1Q encapsulation to configure VLANs instead of a VLAN database. Network automation allows you to configure your devices in a programmatic fashion. This can be done using protocols like NETCONF. NETCONF allows you to push down configurations to your device in a predictable manner using XML and YANG data models.

In this lab, VLANs are created using a router-on-a-stick topology, which utilizes subinterfaces instead of a switch-based VLAN database.

You will write Python code using the `ncclient` library to connect and communicate with the device and configure it automatically. This is how network engineers are automating networks today in the enterprise and cloud.

Environment

This lab was built and developed on a locally hosted DevNet VM/Cisco CSR1000v router on a student workstation. This was to circumvent intermittent connectivity/access problems

encountered with the NDG NetLab environment while developing this lab. The lab can be completed using either of the following:

1. NDG NetLab DevNet environment (provided the proper connectivity/device access is working).

2. Student-hosted environment using a locally installed DevNet VM and Cisco CSR1000v router (provided that the student has completed the prerequisite local setup).

All steps, Python scripts, and verifications are identical. The only things that will change are the IP addressing of the devices, username/passwords, and commands to verify connectivity to your local environment. Students should validate device reachability, NETCONF service availability, and proper credentials for their environment prior to starting the lab.

Topology Description

This lab uses a simple topology consisting of:

- One DevNet virtual machine (student workstation)
- One Cisco CSR1000v router
- Communication over NETCONF (TCP port 830)

The student device will make connections to the router via Python codes. Python codes will be used to push configuration and pull configuration back from the router. The topology for this lab can be seen in Figure 1.

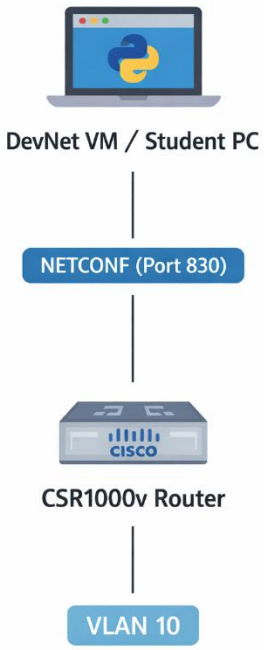

Figure 1. Network Topology for VLAN Automation Lab

This topology maps out how the DevNet VM will communicate with the CSR1000v virtual router via NETCONF over port 830. The DevNet VM will have Python automation codes running to push configuration to the router and then validate the VLAN configuration.

Required Resources

To successfully complete this lab, the following resources are required:

1. **NDG Netlab DevNet or Local Machine with DevNet VM**

   Access to either the NDG Netlab DevNet environment or a locally hosted DevNet VM with a functioning CSR1000v router.

2. **Student Workstation (DevNet VM)**

   A Linux-based virtual machine with terminal access for running Python scripts.

3. **Cisco CSR1000v Router**

   A virtual router instance with NETCONF capability enabled.

4. **Valid Router Credentials**

   Username and password for accessing the CSR1000v router (if needed). If you login to the CSR1000v router and see CSR1kv> as your command prompt, then type enable. If prompted for a username and/or password, enter this lab's login:

```
Username: cisco
Password: cisco
```

Note: Based on the student's specific lab environment, the Username and Password could also be `admin` instead of `cisco`.

Once you have logged in, you have privilege access and will see `CSR1kv#` at the prompt.

5. **Verified Router IP address**

   Once logged in to the CSR1000v router, run the following command:

```
show ip interface brief
```

   Look for an interface with an IP address, such as `GigabitEthernet1` `192.168.56.101`. **This is the IP address that must be used in the Python scripts contained in this lab.**

6. **Verified Connectivity from DevNet VM**

   Once the IP address has been verified on the CSR1000v router, ping the IP address on the DevNet VM terminal with the following command:

```
ping <IP address>
```

   If the router replies, the connection is working. If there is no reply, troubleshoot accordingly.

7. **Python 3.x**

   Installed on the DevNet VM for executing automation scripts.

8. **ncclient Python Library**

   Required for establishing NETCONF sessions and interacting with the router.

   Install using:

   ```
   pip3 install ncclient
   ```

9. **Network Connectivity to CSR1000v**

   The workstation must be able to reach the router over:

   TCP Port **830** (NETCONF)

10. **Text Editor (nano or vi)**

    Used to create and edit Python scripts.

11. **Terminal Access**

    Required to execute commands and run Python scripts.

12. **Basic Networking Knowledge**

    Understanding of VLAN concepts and basic router configuration.

Optional Resources

1. Screenshot tool for capturing lab checkpoints
2. Internet access (if installing packages is required)

Lab Requirements

Ensure the following important steps are completed and in place before starting the lab:

- ✓ Access to NDG Netlab DevNet environment
- ✓ CSR1000v device is powered on, and you are logged in (refer to item number 4 under required resources).
- ✓ Python 3 is installed:
  - In the DevNet VM terminal, run:

    ```
    python3 --version
    ```

- If you see something like 3.x.x- Python is already installed. If Python 3 is not installed, run the following command:

  ```
  sudo apt update
  sudo apt install python3 -y
  ```

- Then, install pip (if needed) by issuing the following command in the DevNet VM terminal: `sudo apt install python3-pip -y`

✓ `ncclient` library is installed using the following command on the DevNet VM terminal:

```
pip3 install ncclient
```

Instructions

Step 1: Verify NETCONF is Enabled

NETCONF operates over TCP port 830, which must be reachable from the DevNet VM.

- Access the CSR1000v router terminal and run:

  ```
  show running-config | include netconf
  ```

  If you see output such as "`netconf-yang`", NETCONF is enabled.

- If NETCONF is not enabled, configure it using the following commands in the router terminal:

  ```
  conf t
  netconf-yang
  end
  ```

Step 2: Create Python Script for VLAN Configuration

- In the DevNet VM terminal, create the following new Python file using nano or vi:

  ```
  nano vlan_config.py
  ```

  Explanation: `vlan_config.py` uses Python/`ncclient` to initiate a NETCONF session to a Cisco CSR1000v router over port 830. The script authenticates against the router using its IP address and hardcoded username/password from the script. After the session has been established the script pushes an XML config payload to the router through NETCONF's edit-config method. This payload creates a subinterface by the name of GigabitEthernet1.10. Since routers commonly use subinterfaces to define VLANs, this automatically maps to VLAN 10 on the router.

This configures 802.1Q encapsulation with VLAN ID 10, configures the IP address 192.168.10.1/24 and provides description for VLAN10_SUBINTERFACE. The router now understands how to associate that subinterface with VLAN 10 and route traffic for that VLAN.

- Insert the following code into your `vlan_config.py` file using the verified IP address gathered previously in the lab. Make any edits needed to the script based on the individual student's lab environment:

```
from ncclient import manager


router = {
    "host": "192.168.56.101",
    "port": 830,
    "username": "cisco",
    "password": "cisco",
    "hostkey_verify": False
}

netconf_config = """
<config>
  <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native">
    <interface>
      <GigabitEthernet>
        <name>1.10</name>
        <encapsulation>
          <dot1Q>
            <vlan-id>10</vlan-id>
          </dot1Q>
        </encapsulation>
        <ip>
          <address>
            <primary>
              <address>192.168.10.1</address>
              <mask>255.255.255.0</mask>
            </primary>
          </address>
        </ip>
        <description>VLAN10_SUBINTERFACE</description>
      </GigabitEthernet>
    </interface>
  </native>
</config>
"""

with manager.connect(**router) as m:
    m.edit_config(target="running", config=netconf_config)
```

```
print("GigabitEthernet1.10 successfully configured for VLAN 10")
```

```python
from ncclient import manager

router = {
    "host": "192.168.56.101",
    "port": 830,
    "username": "cisco",
    "password": "cisco",
    "hostkey_verify": False
}

netconf_config = """
<config>
  <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native">
    <interface>
      <GigabitEthernet>
        <name>1.10</name>
        <encapsulation>
          <dot1Q>
            <vlan-id>10</vlan-id>
          </dot1Q>
        </encapsulation>
        <ip>
          <address>
            <primary>
              <address>192.168.10.1</address>
              <mask>255.255.255.0</mask>
            </primary>
          </address>
        </ip>
        <description>VLAN10_SUBINTERFACE</description>
      </GigabitEthernet>
    </interface>
  </native>
</config>
"""

with manager.connect(**router) as m:
    m.edit_config(target="running", config=netconf_config)
    print("GigabitEthernet1.10 successfully configured for VLAN 10")
```

Figure 2. Content of vlan_config.py file

Step 3: Execute the Configuration Script

In the DevNet VM terminal, run the Python script:

```
python3 vlan_config.py
```

Upon successful execution, the script will display the following confirmation message: `GigabitEthernet1.10 successfully configured for VLAN 10`. Verify the output of the script matches the expected result shown in Figure 3 below:

```
devasc@labvm: ~
File  Edit  View  Search  Terminal  Help
devasc@labvm:~$ python3 vlan_config.py
GigabitEthernet1.10 successfully configured for VLAN 10
devasc@labvm:~$
```

Figure 3. Output result of python3 vlan_config.py script

The output confirms that the NETCONF edit-config operation was successful and the configuration was applied to the router.

Important Note: Once you execute the Python script (vlan_config.py), you may see the following error:

`AuthenticationException: Authentication failed.` On the CSR1000v, you may also see:

```
%DMI-5-AUTHENTICATION_FAILED: Authentication failure for netconf over ssh
```

This is because the router is not configured to authenticate SSH/NETCONF sessions against the local user database. Even if console login is successful, NETCONF relies on:

- SSH access (port 830)
- VTY line configuration
- Local user authentication

There are also situations where an existing environment may have created a user with overlapping config (i.e., both a password and secret set). Either situation would prohibit authentication from working.

To remediate this error, run through the following router configuration steps.
(The following steps for creating a user account, enabling SSH, and enabling NETCONF only need to be done <u>once</u>. Once these configurations are set, both `vlan_config.py` and `verify_vlan.py` will be able to connect to the router. However, <u>if your NetLab 'resets' or you begin a new session</u>, you will have to configure these steps again.)

1) Login to your CSR1000v router and enter privileged mode by typing the following:

```
enable
conf t
```

2) Remove the existing user (if present) and recreate it using the following commands:

```
no username cisco
username cisco privilege 15 password cisco
```

3) Configure SSH access with the following commands:

```
ip domain-name lab.local
crypto key generate rsa modulus 1024
```

If prompted to replace existing keys, type `yes`.

4) Configure VTY Lines for Local Login using the following commands:

```
line vty 0 4
login local
transport input ssh
```

5) Verify NETCONF is enabled with the following command:

```
show running-config | include netconf
```

If no output is returned, enable NETCONF by issuing the following command:

```
netconf-yang
```

6) Save the router configuration by issuing the following commands:

```
end
write memory
```

7) Back in the DevNet terminal, test SSH connectivity by using the following command:

`ssh cisco@<router-ip>` (use your verified router IP gathered earlier in the lab)

If SSH login is successful, NETCONF should also work. If asked, "Are you sure you want to continue connecting?" Type `yes`. Successful SSH login is shown below in Figure 4:

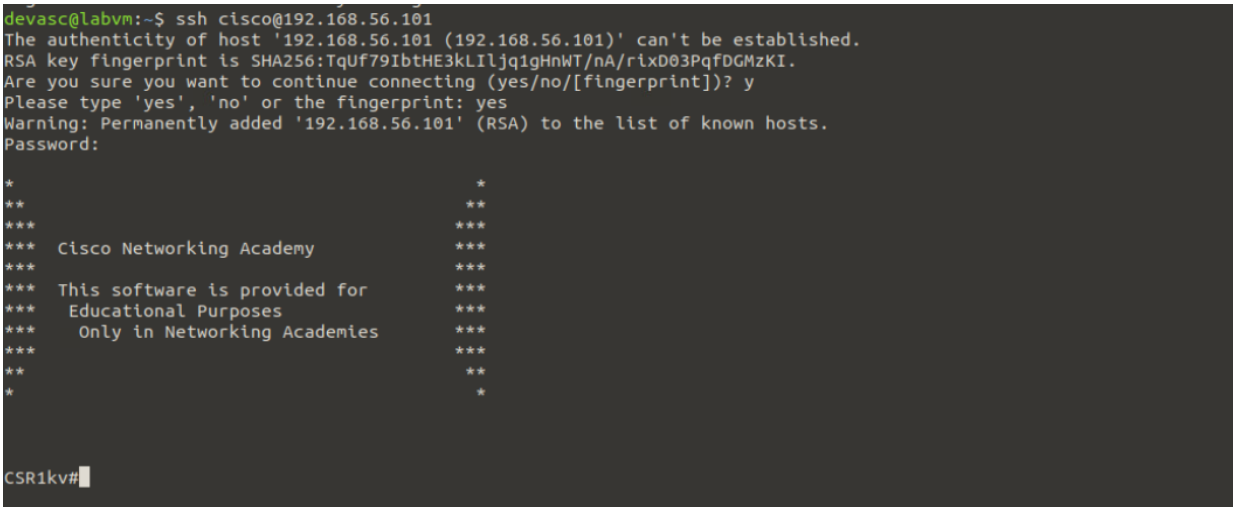

Figure 4. Successful SSH login

8) Once SSH login is confirmed, re-run the Python script:

```
python3 vlan_config.py
```

The script should now be successful and display the following:

```
GigabitEthernet1.10 successfully configured for VLAN 10
```

Step 4: Create Verification Script

- In the DevNet VM terminal, create a second Python file using nano or vi:

```
nano verify_vlan.py
```

Explanation: The `verify_vlan.py` script pulls up the running configuration from the router and verifies that the configuration applied with `vlan_config.py` was successful. This script will not be sending any configuration commands/data to the router. Instead, it will make a read-only NETCONF request to verify the VLAN subinterface was created.

The script connects to the Cisco CSR1000v router using the same NETCONF connection properties (IP address, port 830, username/password) and then sends a NETCONF request asking for the router to return its running configuration.

A filter is used so we only return GigabitEthernet interfaces. This is just to make the output smaller and easier to read. The filter we used will return all data for our configured subinterface (GigabitEthernet1.10).

The XML returned from the router shows us:

- The configured interface `(GigabitEthernet1.10)`
- What VLAN encapsulation is used `(dot1Q VLAN 10)`
- What IP address is configured `(192.168.10.1)`
- The subnet mask
- The interface description

- Insert the following code into the `verify_vlan.py` file, using the verified IP address gathered previously in the lab and edit the script as needed based on the individual student's lab environment:

```
from ncclient import manager

router = {
    "host": "192.168.56.101",
    "port": 830,
    "username": "cisco",
    "password": "cisco",
    "hostkey_verify": False
}

filter_xml = """
<filter>
  <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-
native">
    <interface>
      <GigabitEthernet/>
    </interface>
  </native>
</filter>
"""

with manager.connect(**router) as m:
    result = m.get_config(source="running",
filter=filter_xml)
    print(result.xml)
```

Figure 5. Content of verify_vlan.py file

Step 5: Execute Verification Script

In the DevNet VM terminal, run the Python script:

```
python3 verify_vlan.py
```

XML elements should display the interface name, VLAN ID, IP address, subnet mask, and description. This confirms that the configuration exists in the router's running configuration, as shown below in Figure 6.

```
devasc@labvm:~$ python3 verify_vlan.py
<?xml version="1.0" encoding="UTF-8"?>
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="urn:uuid:d7dbb3ba-1bc0-4d34-84ca-2f786b3290b0" xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0"><data><native xmlns="http://cisco.com/ns/y
ang/Cisco-IOS-XE-native"><interface><GigabitEthernet><name>1</name><ip><address><primary><address>192.168.56.101</address><mask>255.255.255.0</mask></primary></address></ip><mop><enabled>false</enabled><sysid>fa
lse</sysid></mop><negotiation xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-ethernet"><auto>true</auto></negotiation></GigabitEthernet><GigabitEthernet><name>1.10</name><description>VLAN10_SUBINTERFACE</descripti
on><encapsulation><dot1Q><vlan-id>10</vlan-id></dot1Q></encapsulation><ip><address><primary><address>192.168.10.1</address><mask>255.255.255.0</mask></primary></address></ip></GigabitEthernet></interface></nativ
e></data></rpc-reply>
devasc@labvm:~$
```

Figure 6. Output result of python3 vlan_config.py script

Step 6: Verify Using CLI

Access the CSR1000v router terminal and run the following commands:

```
show ip interface brief
show running-config interface GigabitEthernet1.10
```

Explanation: We can use the `show ip interface brief` command to quickly verify that our subinterface GigabitEthernet1.10 exists, has been assigned the correct IP address, and is showing as "Up" status. We can use the `show running-config interface GigabitEthernet1.10` command to view the configuration of our interface which we can use to verify 802.1Q encapsulation along with IP address, subnet mask, and description. These commands verify that our configuration automatically applied correctly and is operating properly. The commands should show the following: GigabitEthernet1.10, encapsulation dot1Q 10, and the ip address and subnet mask. Verify the output of the commands, as shown below in Figure 7:

```
CSR1kv#show running-config interface GigabitEthernet1.10
Building configuration...

Current configuration : 133 bytes
!
interface GigabitEthernet1.10
 description VLAN10_SUBINTERFACE
 encapsulation dot1Q 10
 ip address 192.168.10.1 255.255.255.0
end

CSR1kv#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet1       192.168.56.101  YES NVRAM  up                    up
GigabitEthernet1.10    192.168.10.1    YES other  up                    up
CSR1kv#
```

Figure 7. CLI Verification of GigabitEthernet1.10 Subinterface Configuration.

This dual verification approach (API-based and CLI-based) ensures that the configuration was successfully applied and is operational.

Expected Results

At the end of this lab, the following should be verified:

- Subinterface GigabitEthernet1.10 is configured with VLAN 10 encapsulation (dot1Q)
- IP address 192.168.10.1/24 is assigned
- Description VLAN10_SUBINTERFACE is applied
- Configuration is verified using both NETCONF output and CLI commands

Troubleshooting Tips

The following table outlines common issues students may encounter during the lab and how to resolve them.

Issue	Possible Cause	Solution
Connection failed	Wrong IP address	Verify router IP
Authentication error	Incorrect credentials	Check username/password
NETCONF authentication failed (AuthenticationException)	Router not configured for local SSH authentication OR conflicting user credentials	Reconfigure local user and SSH settings: conf t no username cisco username cisco privilege 15 password cisco ip domain-name lab.local crypto key generate rsa modulus 1024 line vty 0 4 login local transport input ssh end write memory
NETCONF connection works but script fails authentication	SSH not properly configured or VTY lines not using local login	Verify SSH configuration: show running-config | section line vty
NETCONF not working	Not enabled	Enable `netconf-yang`
Script hangs	Port blocked	Confirm port 830 is open
No VLAN output	Script error	Re-run configuration script

Reflection Questions

1. What benefits does NETCONF offer compared to typical CLI configuration?

2. How can consistency in network management be improved with automation?

3. What difficulties did you experience during this lab?

4. Why should you validate after applying configurations?

Reflection Summary

This lab enhances knowledge on network automation through Python and NETCONF, as editing configurations via CLI can be time-consuming. Python provides the ability to enter configurations faster and more efficiently. Automation allows you to make configuration changes in a way that can be repeated while producing the same result every time.